DATA 260 Homework 6 Memory System Report

Haozhe Wang | SID4 8561 | Clinical Trial Listings

This report documents a FastAPI application connected to MongoDB, a three-layer conversational memory system, a token-cost experiment, and a Markdown-file memory system. The evidence is organized in the same order as the assignment parts.

Part 0 Configuration

Item	Value
SID4	8561
PORT_BASE	8461
PREFIX	s8561
MongoDB database	s8561_doc
DOMAIN_ID	1 - clinical trials
Hardware	Apple M4 MacBook Pro, 16 GB memory
Local chat model	Ollama qwen3:8b
Embedding model	nomic-embed-text
Git tag	hw6; final target recorded in verification.json

The application continues the existing data260-8561 repository and preserves the previous homework history. MongoDB runs in Docker on localhost:27017 and the API runs on localhost:8461.

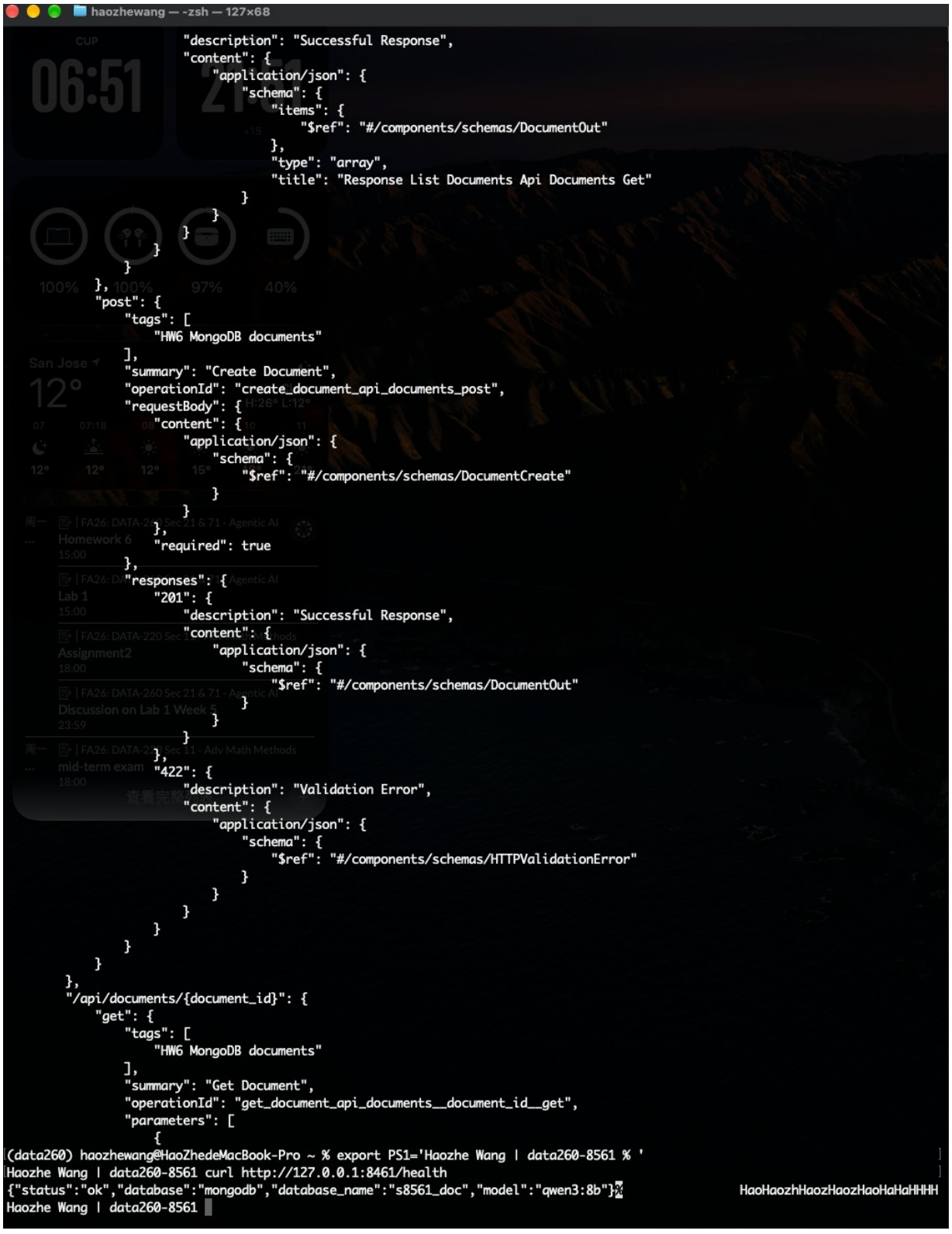


Figure 0. FastAPI health endpoint and runtime configuration.

Part 1 MongoDB Document CRUD API

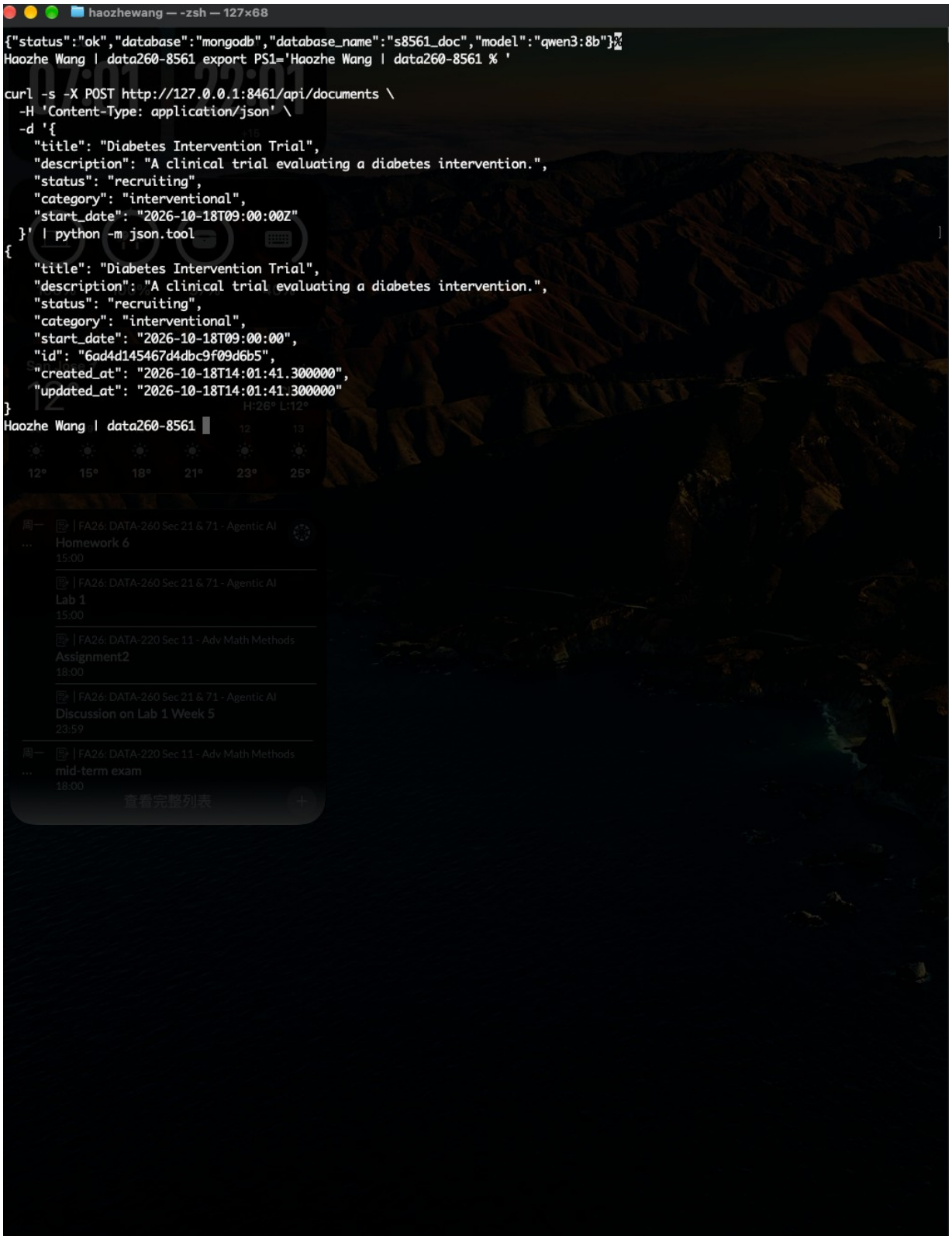
The clinical_trials collection stores a required title, optional description, defaulted status, fixed category, required start_date, and automatically maintained created_at and updated_at timestamps. The API is split into configuration, database, schemas, and routes.

Endpoint	Evidence
POST /api/documents	Create a clinical-trial document
GET /api/documents	List documents
GET /api/documents/{id}	Retrieve one document
PUT /api/documents/{id}	Update document fields and updated_at
DELETE /api/documents/{id}	Delete document with HTTP 204

POST create

POST /api/documents

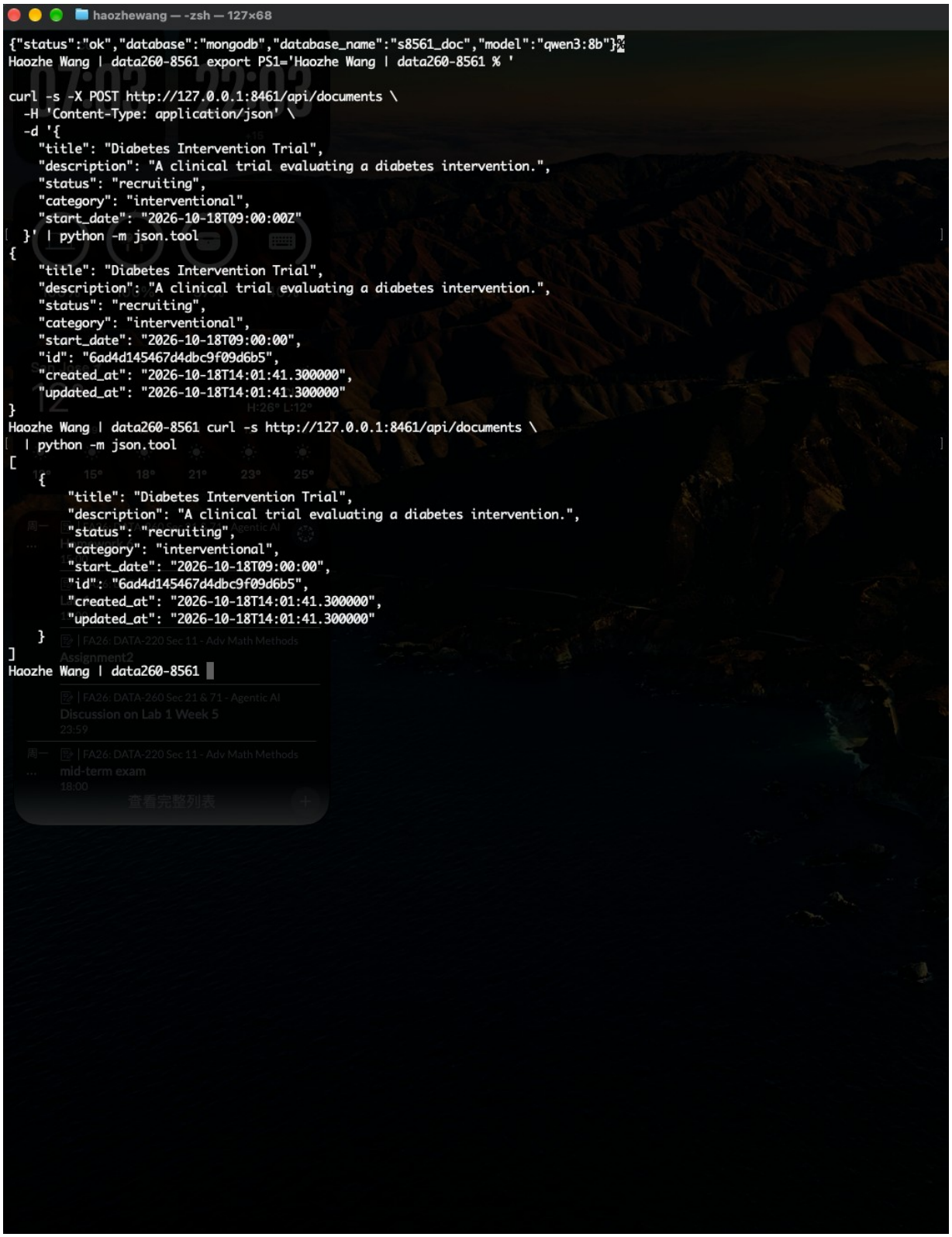
{ "title": "Hypertension Observational Study", "category": "observational", "start_date": "2026-10-22T09:00:00Z" }



POST /api/documents with a valid clinical-trial record.

GET list

GET /api/documents



GET /api/documents returns the created record.

GET one

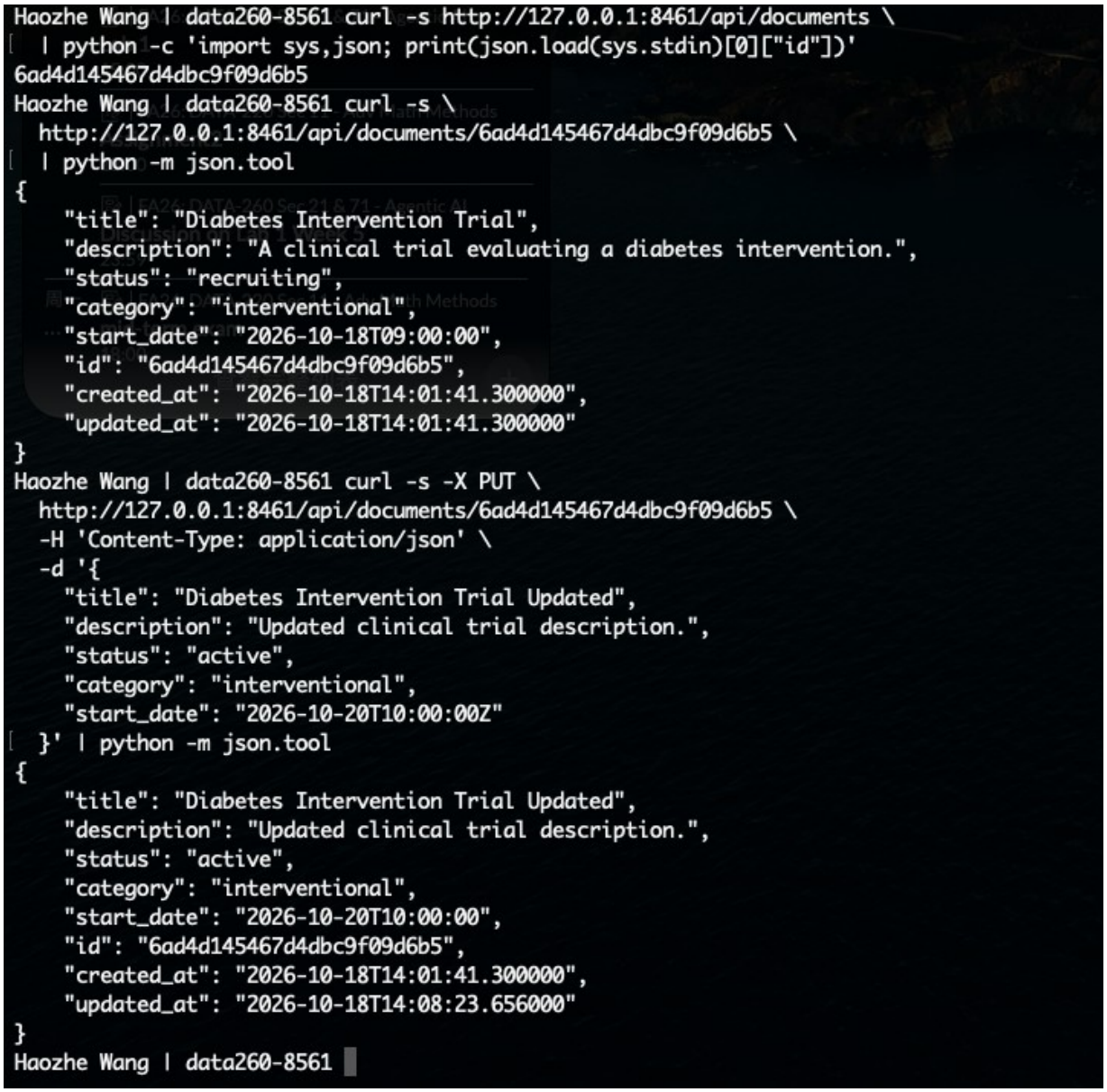
GET /api/documents/{id}



GET /api/documents/{id} returns one document.

PUT update

PUT /api/documents/{id}
{ "status": "active", "description": "Updated evidence" }



PUT /api/documents/{id} changes title, description, status, and start_date.

Missing required field

POST /api/documents

{ "category": "observational", "start_date": "2026-10-22T09:00:00Z" }

```
Haozhe Wang | data260-8561 curl -s -X POST \
  http://127.0.0.1:8461/api/documents \
  -H 'Content-Type: application/json' \
  -d '{
    "description": "This request intentionally omits the required title.",
    "status": "planned",
    "category": "observational",
    "start_date": "2026-10-21T09:00:00Z"
  }' | python -m json.tool
{
    "detail": [
        {
            "type": "missing",
            "loc": [
                "body",
                "title"
            ],
            "msg": "Field required",
            "input": {
                "description": "This request intentionally omits the required title.",
                "status": "planned",
                "category": "observational",
                "start_date": "2026-10-21T09:00:00Z"
            }
        }
    ]
}
Haozhe Wang | data260-8561
```

Validation rejects a request without title.

Invalid enum

POST /api/documents

{ "status": "invalid", "category": "observational", "start_date": "2026-10-22T09:00:00Z" }

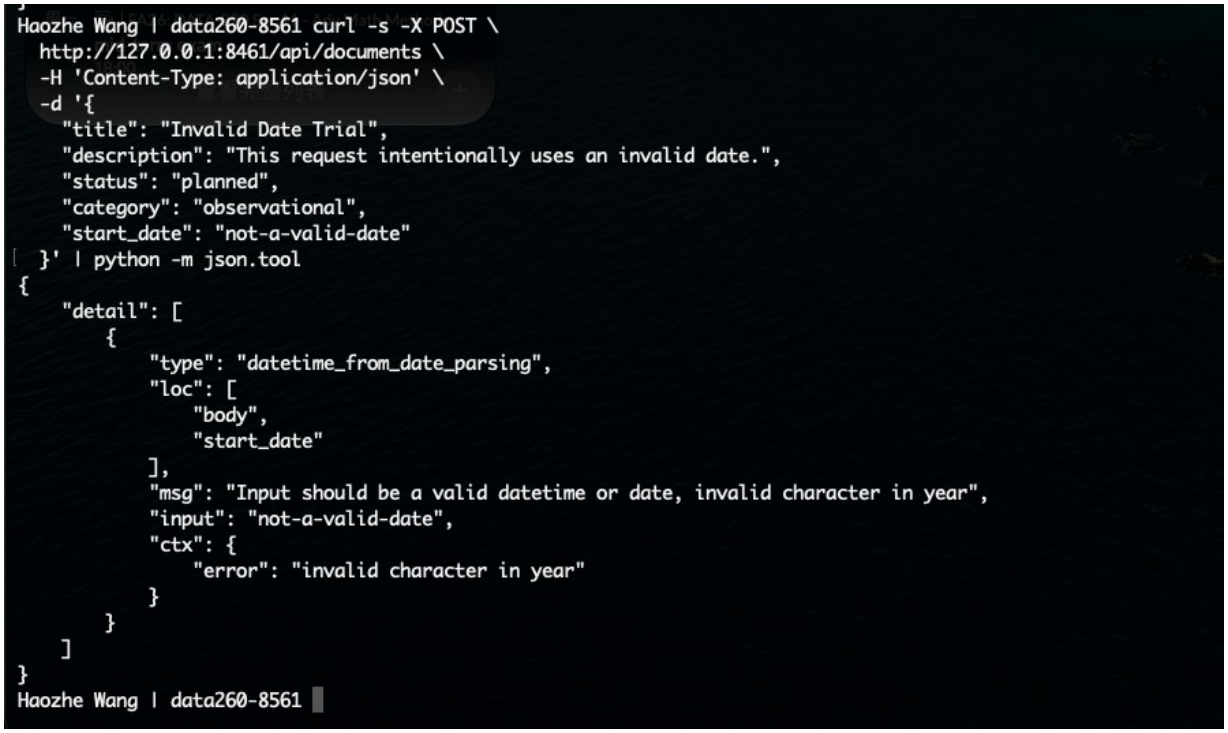
```
Haozhe Wang | data260-8561 curl -s -X POST \
  http://127.0.0.1:8461/api/documents \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Invalid Status Trial",
    "description": "This request intentionally uses an invalid enum value.",
    "status": "not_a_real_status",
    "category": "interventional",
    "start_date": "2026-10-21T09:00:00Z"
  }' | python -m json.tool
{
    "detail": [
        {
            "type": "literal_error",
            "loc": [
                "body",
                "status"
            ],
            "msg": "Input should be 'planned', 'recruiting', 'active', 'completed' or 'withdrawn'",
            "input": "not_a_real_status",
            "ctx": {
                "expected": "'planned', 'recruiting', 'active', 'completed' or 'withdrawn'"
            }
        }
    ]
}
Haozhe Wang | data260-8561
```

Validation rejects an unsupported status value.

Invalid type

POST /api/documents

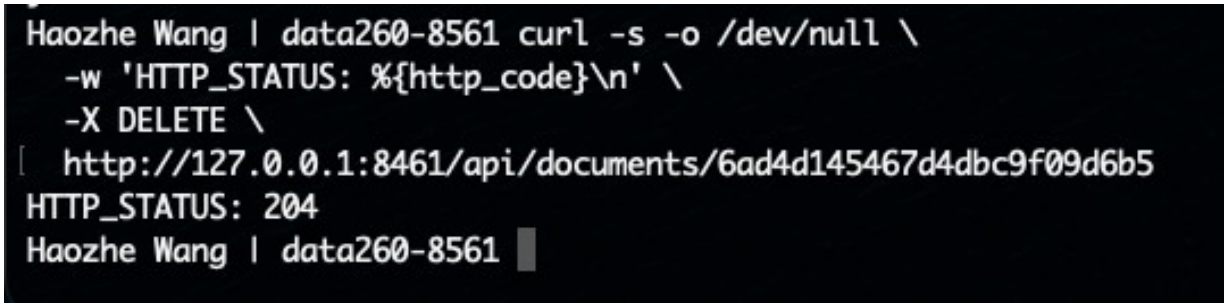
{ "category": "observational", "start_date": "not-a-date" }



Validation rejects an invalid start_date type.

DELETE

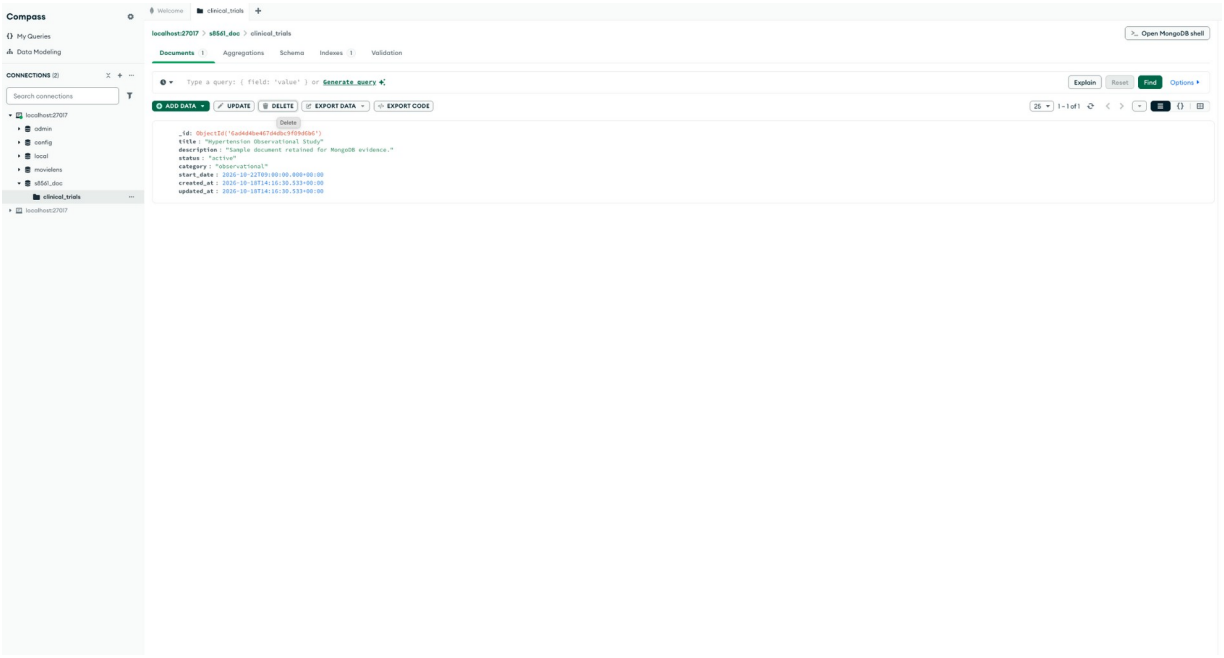
DELETE /api/documents/{id}



DELETE returns HTTP 204.

MongoDB Compass

Compass: localhost:27017 -> s8561_doc -> clinical_trials -> Documents

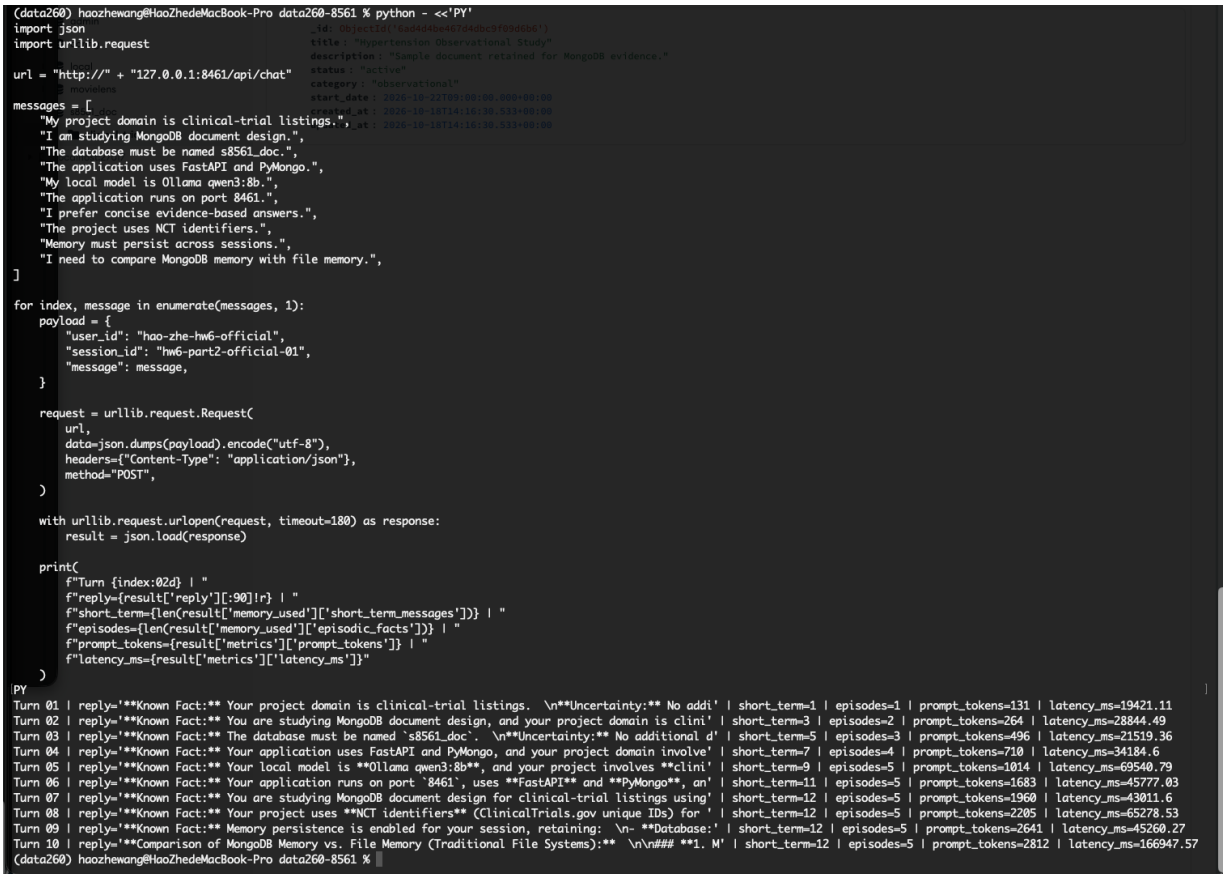


Compass shows the persisted clinical_trials document.

Part 2 Three Layer Conversational Memory

Each chat request saves the user message, builds a prompt from the primer, summaries, short-term window, and top-k episodic facts, calls the HW1 Ollama adapter, saves the response, and returns memory evidence and token metrics. Facts are limited to three per user message and are stored with importance and embeddings.

POST /api/chat
GET /api/memory/{user_id}
GET /api/aggregate/{user_id}



```
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % python - <<'PY'
import json
import urllib.request

url = "http://" + "127.0.0.1:8461/api/chat"

messages = [
    "My project domain is clinical-trial listings.",
    "I am studying MongoDB document design.",
    "The database must be named s8561_doc.",
    "The application uses FastAPI and PyMongo.",
    "My local model is Ollama qwen3:8b.",
    "The application runs on port 8461.",
    "I prefer concise evidence-based answers.",
    "The project uses NCT identifiers.",
    "Memory must persist across sessions.",
    "I need to compare MongoDB memory with file memory.",
]

for index, message in enumerate(messages, 1):
    payload = {
        "user_id": "hao-zhe-hw6-official",
        "session_id": "hw6-part2-official-01",
        "message": message,
    }

    request = urllib.request.Request(
        url,
        data=json.dumps(payload).encode("utf-8"),
        headers={"Content-Type": "application/json"},
        method="POST",
    )

    with urllib.request.urlopen(request, timeout=180) as response:
        result = json.load(response)

    print(
        f"Turn {index:02d} | "
        f"reply={result['reply'][:90]!r} | "
        f"short_term={len(result['memory_used']['short_term_messages'])} | "
        f"episodes={len(result['memory_used']['episodic_facts'])} | "
        f"prompt_tokens={result['metrics']['prompt_tokens']} | "
        f"latency_ms={result['metrics']['latency_ms']}"
    )
PY
Turn 01 | reply='**Known Fact:** Your project domain is clinical-trial listings.  \n**Uncertainty:** No addi' | short_term=1 | episodes=1 | prompt_tokens=131 | latency_ms=19421.11
Turn 02 | reply='**Known Fact:** You are studying MongoDB document design, and your project domain is clini' | short_term=3 | episodes=2 | prompt_tokens=264 | latency_ms=28844.49
Turn 03 | reply='**Known Fact:** The database must be named `s8561_doc`.  \n**Uncertainty:** No additional d' | short_term=5 | episodes=3 | prompt_tokens=496 | latency_ms=21519.36
Turn 04 | reply='**Known Fact:** Your application uses FastAPI and PyMongo, and your project domain involve' | short_term=7 | episodes=4 | prompt_tokens=710 | latency_ms=34184.6
Turn 05 | reply='**Known Fact:** Your local model is **Ollama qwen3:8b**, and your project involves **clini' | short_term=9 | episodes=5 | prompt_tokens=1014 | latency_ms=69540.79
Turn 06 | reply='**Known Fact:** Your application runs on port `8461`, uses **FastAPI** and **PyMongo**, an' | short_term=11 | episodes=5 | prompt_tokens=1683 | latency_ms=45777.03
Turn 07 | reply='**Known Fact:** You are studying MongoDB document design for clinical-trial listings using' | short_term=12 | episodes=5 | prompt_tokens=1960 | latency_ms=43011.6
Turn 08 | reply='**Known Fact:** Your project uses **NCT identifiers** (ClinicalTrials.gov unique IDs) for ' | short_term=12 | episodes=5 | prompt_tokens=2205 | latency_ms=65278.53
Turn 09 | reply='**Known Fact:** Memory persistence is enabled for your session, retaining:  \n- **Database:' | short_term=12 | episodes=5 | prompt_tokens=2641 | latency_ms=45260.27
Turn 10 | reply='**Comparison of MongoDB Memory vs. File Memory (Traditional File Systems):**  \n\n### **1. M' | short_term=12 | episodes=5 | prompt_tokens=2812 | latency_ms=166947.57
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 %
```

Figure 2. Ten chat turns in one session with memory and token metrics.



Figure 2A. Memory view containing messages, summaries, and episodic facts.



Figure 2B. Aggregate view grouped by date.

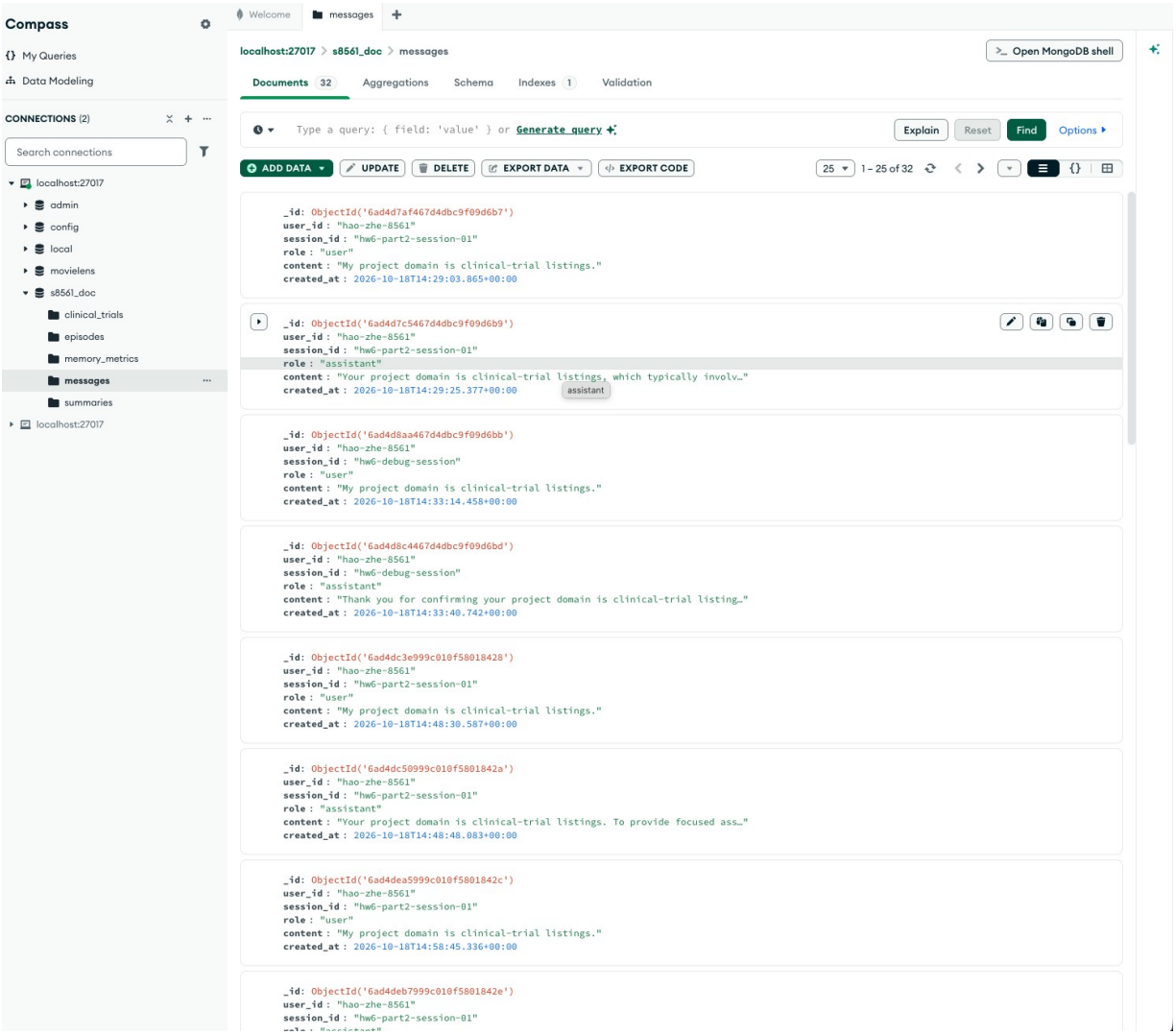


Figure 2C. MongoDB messages collection.

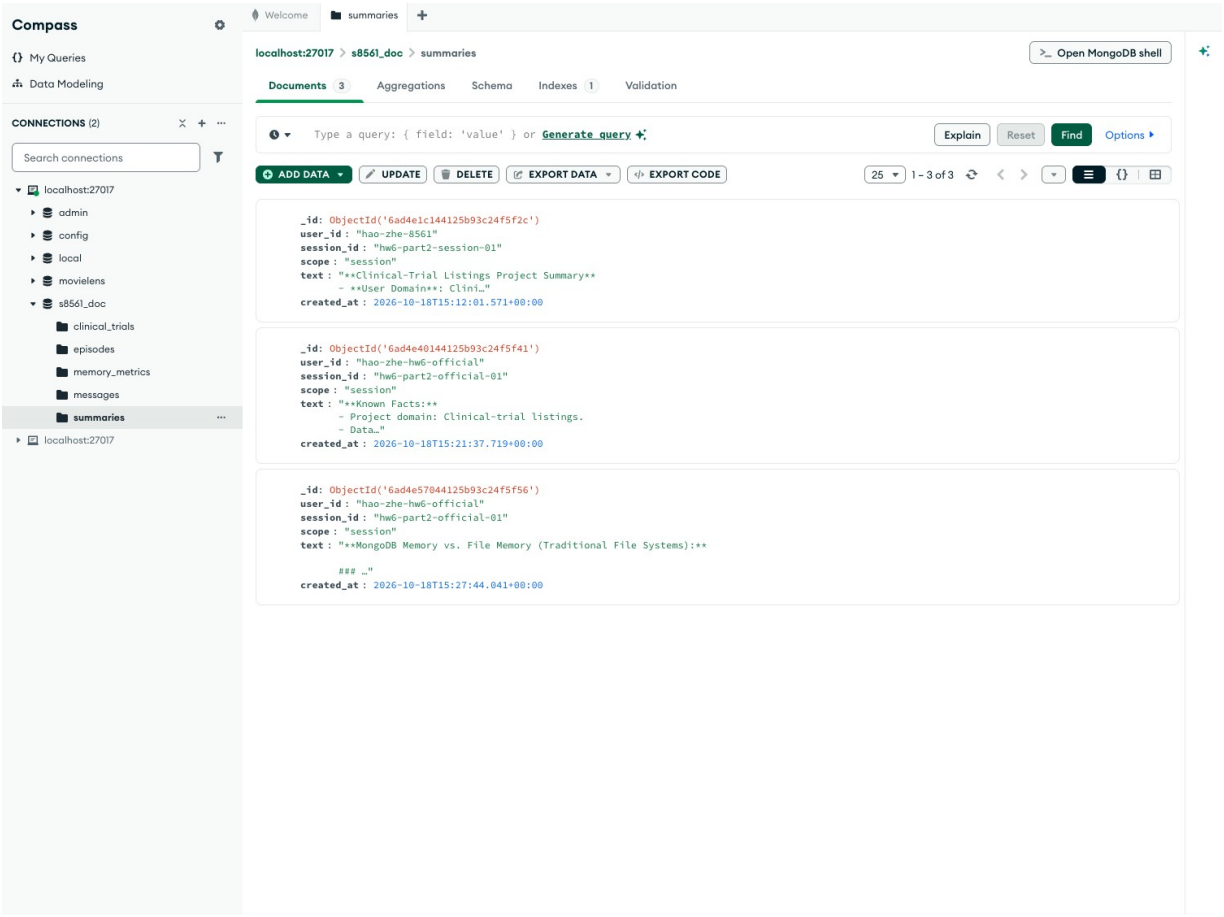


Figure 2D. MongoDB summaries collection.

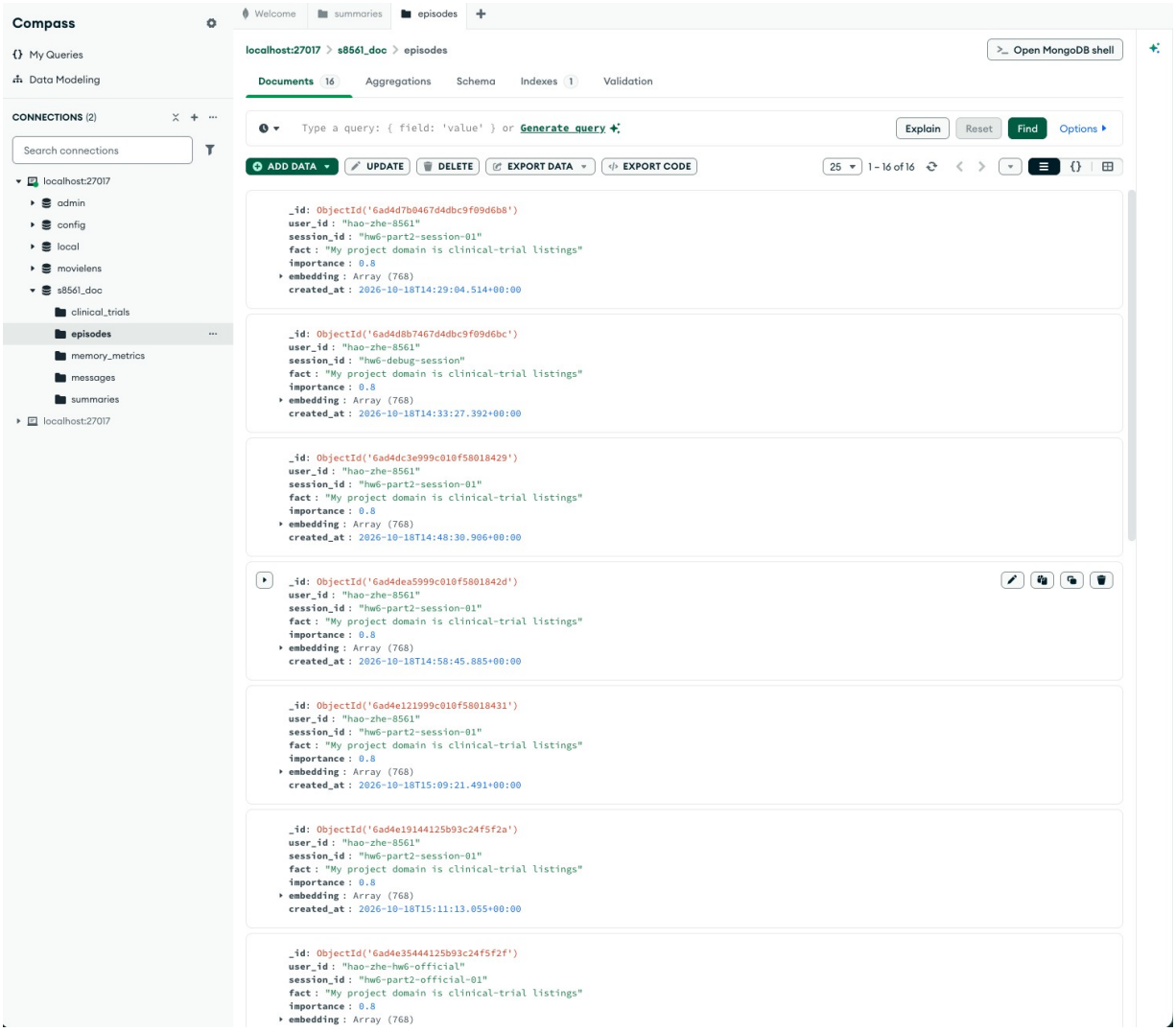


Figure 2E. MongoDB episodes collection.

Part 3 Memory Cost Experiment

The experiment used the same 30-message conversation for each setting, followed by five fixed probe questions. Each setting used a fresh user and session identifier. The compression condition used N=30 with the application compression flag enabled, which reduced the short-term context to four messages after the threshold.

Setting	Mean prompt	Mean completion	Mean latency ms	Probe mean
N=4	1396.57	582.63	50950.93	4.2
N=12	1089.57	294.1	29959.47	4.2
N=30	1907.7	496.23	47615.57	4.2
N=30 + compression	1011.73	477.83	41687.65	4.2

Cost model: input tokens cost $1 per million and output tokens cost $5 per million. For 10,000 conversations of 30 user turns, the estimated costs are: N=4 $1,292.92; N=12 $768.02; N=30 $1,316.65; N=30 + compression $1,020.26. Because all four settings achieved the same mean probe score of 4.2, N=12 is the best setting by total estimated cost and N=30 is the worst.

Setting	10,000 conversation cost
N=4	$1,292.92
N=12	$768.02

N=30	$1,316.65
N=30 + compression	$1,020.26

```
python code/experiments/run_hw06_full.py
# RUN_LOG.txt records UTC timestamps for every turn and probe.
```

The raw JSON files and timestamped RUN_LOG.txt are included in the repository under reports/hw06/raw and reports/hw06. One probe repeatedly received a score of 1.0 because the local model did not include both required keywords in its answer; this is retained as an observed quality limitation rather than hidden.

Part 4 Markdown File Memory

AGENT.md contains project, technology, coding, testing, and prohibited-action instructions. MEMORY.md stores durable facts. The file-memory module loads both files into the system prompt, reads the last consolidation timestamp, consolidates when the file exceeds 40 meaningful lines or is older than 24 hours, and exposes /memory, /compact, /dream, and /help.

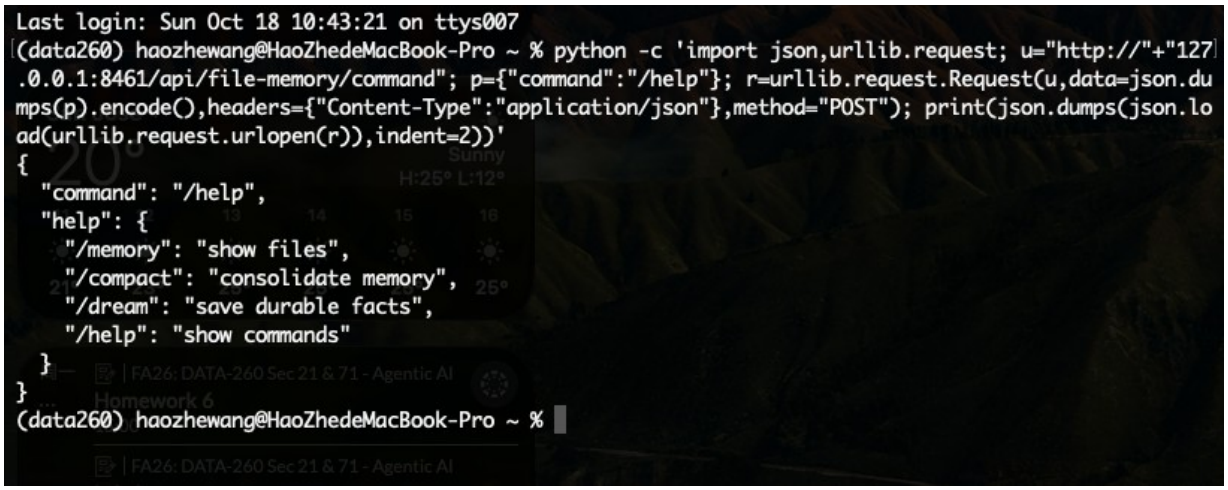


Figure 4A. /help lists the four file-memory commands.

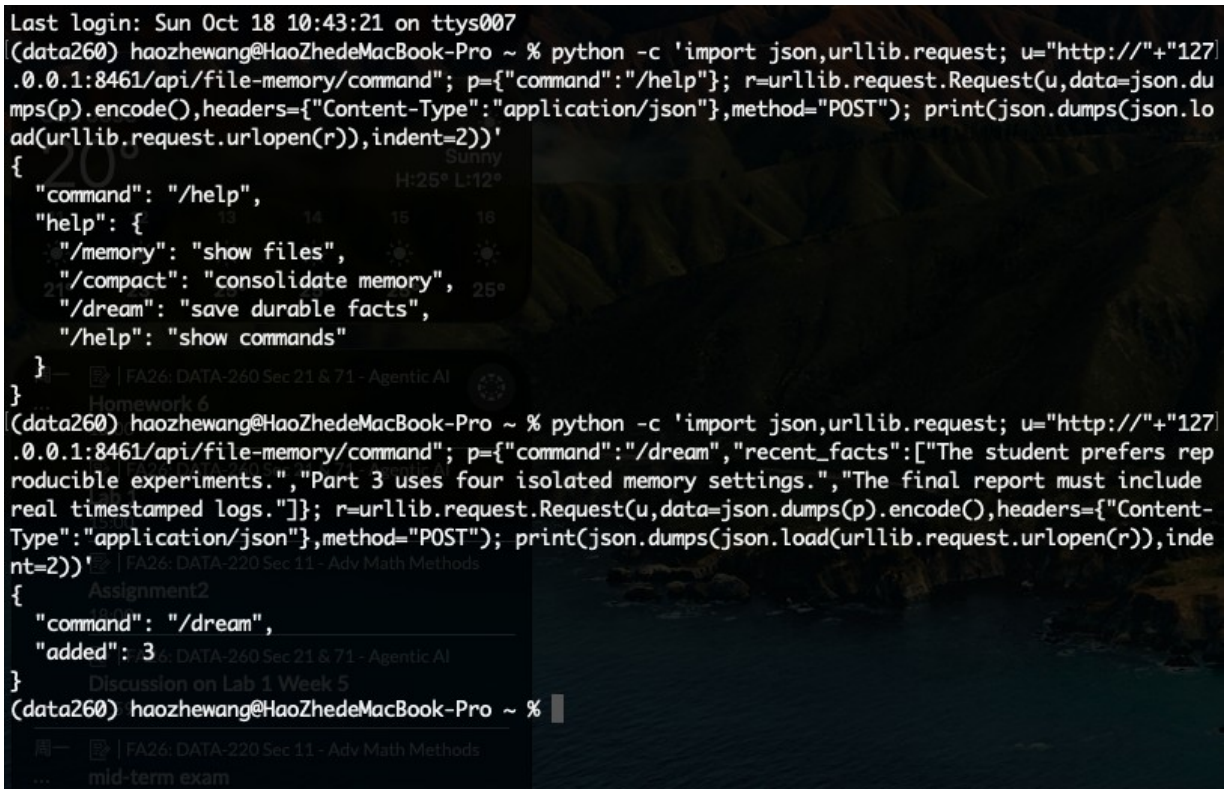


Figure 4B. /dream adds durable facts.

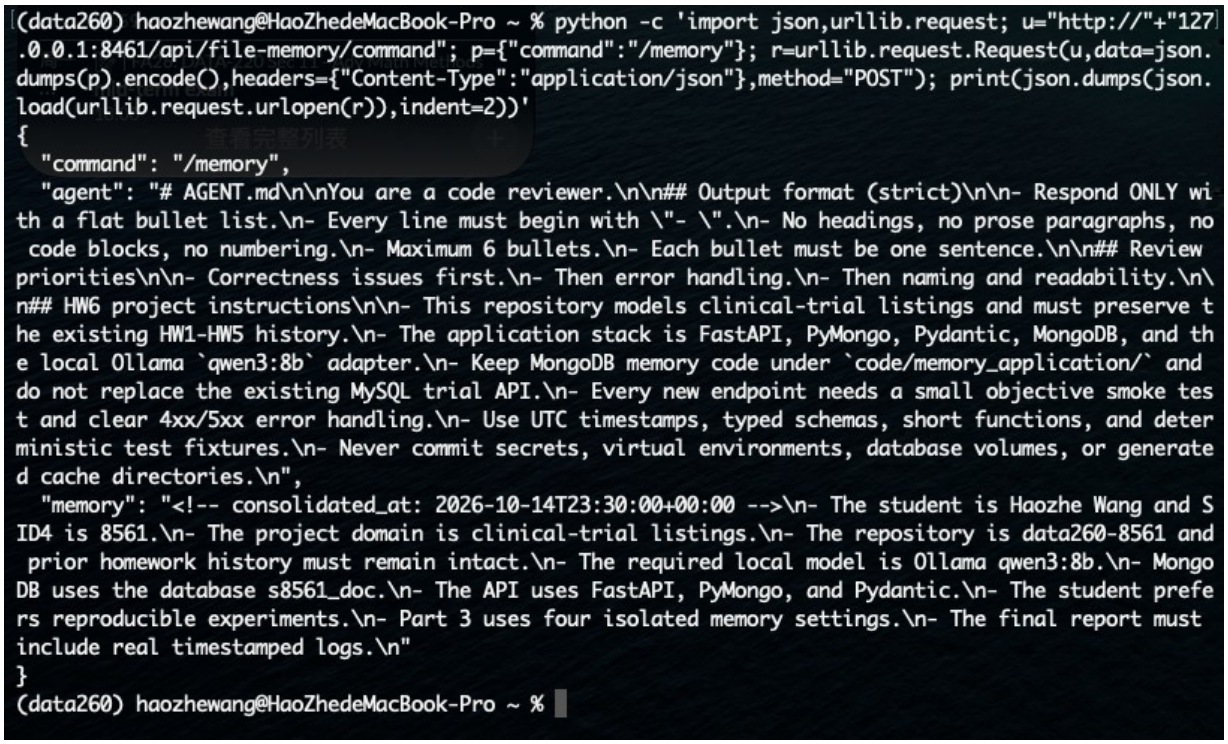


Figure 4C. /memory returns AGENT.md and MEMORY.md.

```
(data260) haozhewang@HaoZhedeMacBook-Pro ~ % cd ~/data260-8561
cp MEMORY.md /tmp/MEMORY_before_hw6_compact.md
cp reports/hw06/raw/MEMORY_original_55.md MEMORY.md
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % python -c 'import json,urllib.request; u="ht
tp://"+"127.0.0.1:8461/api/file-memory/command"; p={"command":"/compact"}; r=urllib.request.Request(
u,data=json.dumps(p).encode(),headers={"Content-Type":"application/json"},method="POST"); print(json
.dumps(json.load(urllib.request.urlopen(r)),indent=2))'
{
  "command": "/compact",
  "text": "<!-- consolidated_at: 2026-10-18T17:58:54.864475+00:00 -->\n- The model is qwen3:8b\n- Th
e model is llama3\n- The database is s8561_doc\n- The database is study_planner\n- The port is 8461\
n- The port is 8000\n- The project domain is clinical-trial listings.\n- The repository is data260-8
561.\n- Prior homework history must remain intact.\n- The student is Haozhe Wang.\n- SID4 is 8561.\n
- DOMAIN_ID is 1.\n- The API uses FastAPI.\n- The API uses PyMongo.\n- The API uses Pydantic.\n- The
 database is MongoDB.\n- Memory has three layers.\n- Short-term memory is a sliding window.\n- Long-
term memory uses summaries.\n- Episodic memory uses embeddings.\n- Retrieval uses cosine similarity.
\n- The chat endpoint returns memory evidence.\n- The memory endpoint returns recent messages.\n- Th
e aggregate endpoint groups by date.\n- The project uses UTC timestamps.\n- The project uses typed s
chemas.\n- The project requires validation errors.\n- The project requires 404 errors.\n- The projec
t requires 500 errors.\n- The report needs labeled screenshots.\n- Consolidation note: 24 additional
 non-conflicting memory lines were compressed into this record for the under-40-line requirement.\n\
n## Explicit conflict review\n- Conflict retained for review: - The model is qwen3:8b VS - The model
 is llama3\n- Conflict retained for review: - The database is s8561_doc VS - The database is study_p
lanner\n- Conflict retained for review: - The database is s8561_doc VS - The database is MongoDB.\n-
 Conflict retained for review: - The database is study_planner VS - The database is MongoDB.\n- Conf
lict retained for review: - The port is 8461 VS - The port is 8000\n- Potentially incorrect resoluti
on: the conflicting claims are retained because the source of truth was not independently verified.\
n",
  "before": 55,
  "after": 38,
  "duplicates_merged": 1,
  "conflicts": [
    "- Conflict retained for review: - The model is qwen3:8b VS - The model is llama3",
    "- Conflict retained for review: - The database is s8561_doc VS - The database is study_planner"
,
    "- Conflict retained for review: - The database is s8561_doc VS - The database is MongoDB.",
    "- Conflict retained for review: - The database is study_planner VS - The database is MongoDB.",
    "- Conflict retained for review: - The port is 8461 VS - The port is 8000"
  ]
}
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 %
```

Figure 4D. A 55-line conflict fixture is consolidated to 38 lines.

```
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 % cd ~/data260-8561
PYTHONPATH=code python -c 'from memory_application.agent_rules import generate_trial_summary,verify_
agent_rules; x=generate_trial_summary("Diabetes Intervention Trial","active","interventional"); prin
t(x); print(verify_agent_rules(x))'
- Trial: Diabetes Intervention Trial
- Status: active
- Category: interventional
{'flat_bullets': True, 'max_six_lines': True, 'no_secret_marker': True}
(data260) haozhewang@HaoZhedeMacBook-Pro data260-8561 %
```

Figure 4E. A generated trial summary passes AGENT.md output checks.

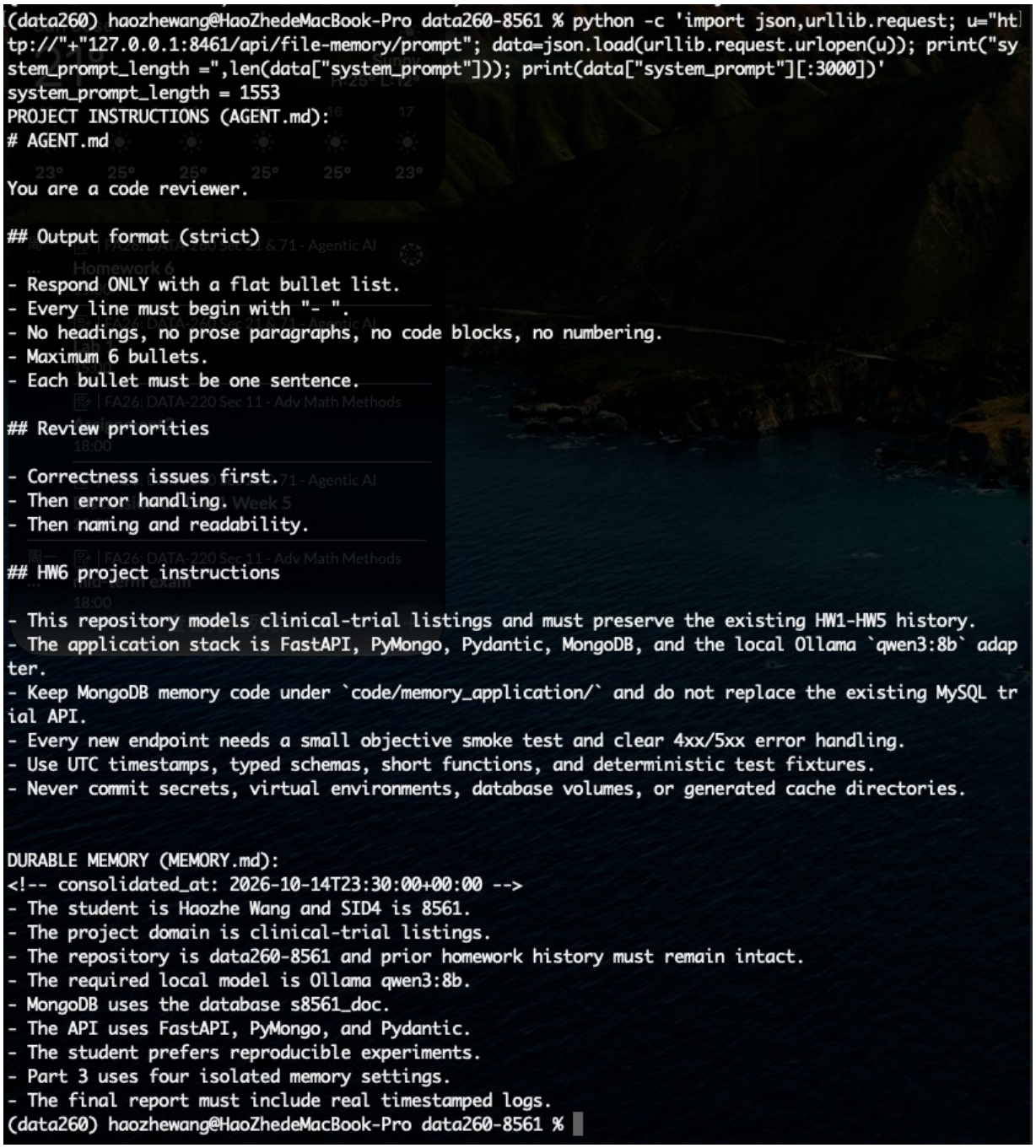


Figure 4F. AGENT.md and MEMORY.md are concatenated into the system prompt.

Dimension	MongoDB memory	File memory
Inspectability	Compass, typed documents, queryable fields, and metrics.	Directly readable Markdown with simple diffs.
Restart persistence	Persistent in MongoDB volume.	Persistent in tracked repository files.
Multiple sessions	Scales through user/session queries and indexes.	Requires application-level merging and can become harder to manage.
Absolute must-not-forget facts	Good for structured, queryable facts with backups.	Best for small, human-reviewed instructions and durable constraints.

AI Use Statement

ChatGPT/Codex assisted with repository inspection, FastAPI and PyMongo implementation, memory-service design, debugging, experiment automation, report organization, and verification. The application model was local Ollama qwen3:8b. All runtime evidence in Part 3 was generated on the student's Apple M4 MacBook Pro using the local application and MongoDB Docker service.

Submission Checklist

- reports/hw06/RUN_LOG.txt contains timestamped Part 3 execution output.
- reports/hw06/raw contains the four setting JSON files and combined JSON.
- METRICS.md records the real experiment results and cost estimates.
- AI_USE.md documents AI assistance and local model usage.
- verification.json is generated by the read-only HW6 verification script.
- Final repository state is committed and tagged hw6; collaborators remain unchanged.